calculation capabilities of Julia. But here we will talk about how Julia is not only capable of working on complex scientific calculations efficiently but also on commercial-based problems, and can tackle Python in machine learning and neural networks.

Objective and experimentation

Julia is as simple as Python but is a compiled language like C. So let's test how fast Julia is in comparison with Python. For that, we will first test these languages on some simple programs and then move to the main focus of our experiment, which is to test them for machine learning and deep learning.

Julia and Python provide many libraries and open source benchmarking tools. For benchmarking and calculating time in Julia, we used CPUTime and time libraries. Similarly, for Python, we used the time module.

Matrix multiplication

We tried out simple arithmetic operations first, but since these will not generate much difference in time we decided to check out the timing in matrix multiplication. We started with creating two (10 * 10) matrices of random float numbers and performed dot products in these. As we know, Python has a NumPy library, which is famous for working with matrices and vectors. Similarly, Julia has a LinearAlgebra library that works well with matrices and vectors. So we compared the matrix multiplication with and without using their respective libraries. The source code for all the programs used in this article is available in our GitHub repository *(https://github.com/mr-nerdster/Julia_Research.gitsee)*. The 10x10 matrix multiplication program written in Julia is given below:

```
@time LinearAlgebra.mul!(c,x,y)

function MM()
    x = rand(Float64,(10,10))
    y = rand(Float64,(10,10))
    c = zeros(10,10)
    for i in range(1,10)
        for j in range(1,10)
            for k in range(1,10)
                c[i,j] += x[i,k]*y[k,j]
            end
        end
    end
end
@time MM

0.000001 seconds
MM (generic function with 1 method)
```

Julia takes 0.000017 seconds using the library and 0.000001 seconds using loops.

The same matrix multiplication program was written in Python, as shown below. From the results it can be seen that with the library the program takes less time compared to without the library.

```
import numpy as np
import time as t
x = np.random.rand(10,10)
y = np.random.rand(10,10)
start = t.time()
z = np.dot(x, y)
print("Time = ",t.time()-start)
Time =  0.001316070556640625

import random
import time as t
l = 0
h= 10
cols = 10
rows= 10

choices = list (map(float, range(l,h)))
x = [random.choices (choices , k=cols)
for _ in range(rows)]
y = [random.choices (choices , k=cols)
for _ in range(rows)]

result = [([0]*cols) for i in range
(rows)]

start = t.time()

for i in range(len(x)):
    for j in range(len(y[0])):
        for k in range(len(result)):
            result[i][j] += x[i][k] *
y[k][j]

print(result)
print("Time = ", t.time()-start)

Time =  0.0015912055969238281
```

Python takes 0.0013 seconds using the library and 0.0015 seconds using loops.

Linear search

The next experiment that we performed was a linear search on one hundred thousand randomly generated numbers. We used two methods here — one by using a *for* loop and the other by using an operator. We performed 1000 searches with integers ranging from 1 to 1000, and as you can see in the output below we also printed out how many integers we find in the data set. The output of time by using loops and by using the IN operator is given below. Here we measured time by taking the median CPU time of 3 runs.

The program was written for Julia and the results are shown below.

```
FOR_SEARCH:
       Elapsed CPU time: 0.417767
seconds
           matches: 550
       Elapsed CPU time: 0.41119
seconds
           matches: 550
       Elapsed CPU time: 0.425561
seconds
           matches: 550

IN:
       Elapsed CPU time: 0.433371
seconds
           matches: 550
       Elapsed CPU time: 0.44637
seconds
           matches: 550
       Elapsed CPU time: 0.433301
seconds
           matches: 550
```